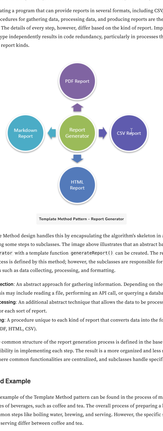
Template Method Pattern

The template method is a behavioral design pattern that defines the skeleton of an algorithm in the superclass but allows subclasses to change specific steps without changing the algorithm's structure. Put differently, it acts as an algorithm's template, letting subclasses take over specific steps while maintaining its general structure.

The pattern adheres to the "Don't call us, we'll call you" principle, in which the subclasses are assigned specific tasks by the superclass, which contains the fixed steps of the algorithm.

Consider creating a program that can provide reports in several formats, including CSV, HTML, and PDF. The procedures for gathering data, processing data, and producing reports are the same for all report types. The details vary for each report type, however. Based on the kind of report, implementing each report type independently results in code redundancy, particularly in processes that are shared by all report kinds.



Template Method Pattern – Report Generator

The Template Method design handles this by encapsulating the algorithm's skeleton in a method and delegating some steps to subclasses. The image above illustrates that an abstract base class `ReportGenerator` with a template method `generateReport()` can be created. The report creation process is defined by this method; however, the subclasses are responsible for carrying out specific tasks such as data collecting, processing, and formatting.

- **Data Collection**: An abstract approach for gathering information. Depending on the type of report, this may include reading a file, performing an API call, or querying a database.
- **Data processing**: An additional abstract technique that allows the data to be processed as needed for each sort of report.
- **Formatting**: A procedure unique to each kind of report that converts data into the format of choice (PDF, HTML, CSV).

This way, the common structure of the report generation process is defined in the base class while allowing flexibility in implementing each step. The result is a more organized and less redundant codebase, where common functionalities are centralized, and subclasses handle specific details.

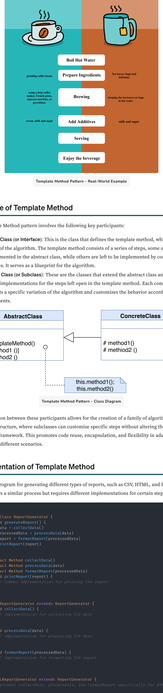
Real-World Example

A real-world example of the Template Method pattern can be found in the process of making different types of beverages, such as coffee and tea. The overall process of preparing a hot beverage involves common steps like boiling water, brewing, and serving. However, the specific steps for brewing and serving differ between coffee and tea.

In this scenario:

- The abstract class represents the generic process of making a hot beverage. It includes a template method that outlines the overall steps like boiling water, brewing, and serving.
- The concrete classes, Coffee and Tea, extend the abstract class. They provide specific implementations for the steps like brewing and serving, which are unique to each type of beverage.

The illustration below shows two parts, one for coffee and one for tea. The steps which are common for both are shown in the center line and those that vary for the two beverages are shown in their specific parts.

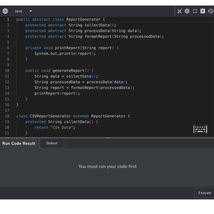


Template Method Pattern – Real-World Example

Structure of Template Method

The Template Method pattern involves the following key participants:

1. **Abstract Class (or Interface)**: This is the class that defines the template method, which is the skeleton of the algorithm. The template method consists of a series of steps, some of which may be implemented in the abstract class, while others are left to be implemented by concrete subclasses. It serves as a blueprint for the algorithm.
2. **Concrete Class (or Subclass)**: These are the classes that extend the abstract class and provide concrete implementations for the steps left open in the template method. Each concrete class represents a specific variation of the algorithm and customizes the behavior according to its requirements.



Template Method Pattern – Class Diagram

The interaction between these participants allows for the creation of a family of algorithms sharing a common structure, where subclasses can customize specific steps without altering the overall algorithmic framework. This promotes code reuse, encapsulation, and flexibility in adapting the algorithm to different scenarios.

Implementation of Template Method

Consider a program for generating different types of reports, such as CSV, HTML, and PDF. Each report follows a similar process but requires different implementations for certain steps.

```
abstract class ReportGenerator {
    final void generateReport() {
        data = collectData();
        processedData = processData(data);
        report = formatReport(processedData);
        printReport(report);
    }

    abstract Method collectData();
    abstract Method processData(data);
    abstract Method formatReport(processedData)
    Method printReport(report) {
        // common implementation for printing the report
    }
}

class CSVReportGenerator extends ReportGenerator {
    Method collectData() {
        // CSV implementation for collecting CSV data
    }

    Method processData(data) {
        // CSV implementation for processing CSV data
    }

    Method formatReport(processedData) {
        // CSV implementation for formatting CSV report
    }
}

class HTMLReportGenerator extends ReportGenerator {
    // Implement collectData, processData, and formatReport specifically for HTML
}

class PDFReportGenerator extends ReportGenerator {
    // Implement collectData, processData, and formatReport specifically for PDF
}

// Usage code
Main() {
    csvReportGenerator = new CSVReportGenerator()
    csvReportGenerator.generateReport()

    htmlReportGenerator = new HTMLReportGenerator()
    htmlReportGenerator.generateReport()

    pdfReportGenerator = new PDFReportGenerator()
    pdfReportGenerator.generateReport()
}
```

- **ReportGenerator (Abstract Class)**: Defines the skeleton of the report generation algorithm in the `generateReport()` method. This method calls abstract methods (`collectData`, `processData`, `formatReport`) and a concrete method (`printReport`).
- **Concrete Classes (CSVReportGenerator, HTMLReportGenerator, PDFReportGenerator)**: Implement the abstract methods defined in `ReportGenerator`. Each class provides a specific implementation for collecting data, processing it, and formatting it according to the report type (CSV, HTML, or PDF).
- **printReport (Concrete Method)**: Provides a common implementation for printing the report, used by all subclasses.

This structure allows for defining the common steps of report generation in the base class while enabling subclasses to implement specific behavior for different report types. It reduces redundancy and allows for extending the system with new report types easily.

Implementation

```
Java
public abstract class ReportGenerator {
    protected abstract String collectData();
    protected abstract String processData(String data);
    protected abstract String formatReport(String processedData);

    private void printReport(String report) {
        System.out.println(report);
    }

    public void generateReport() {
        String data = collectData();
        String processedData = processData(data);
        String report = formatReport(processedData);
        printReport(report);
    }
}

class CSVReportGenerator extends ReportGenerator {
    protected String collectData() {
        return "CSV Data";
    }
```

Run Code Result | Output

You must run your code first

Execute

Application of Template Method

Some notable applications include:

1. Software Frameworks:

Graphical User Interface (GUI) Frameworks: Template Method is often employed in GUI frameworks where the overall structure of handling user inputs and events is predefined, but the specific behavior may vary in different applications or components.

Web Frameworks: Web frameworks utilize Template Method to provide a structured approach for handling HTTP requests and responses. The overall request lifecycle remains consistent, but developers can customize specific steps based on application requirements.

2. Algorithm Design:

Sorting Algorithms: Template Method can be applied to sorting algorithms. The overall sorting process remains the same, but specific steps like element comparison or swapping may vary depending on the sorting algorithm (e.g., quicksort, mergesort).

Search Algorithms: Similar to sorting, search algorithms can benefit from the Template Method pattern. The search process structure remains consistent, while specific steps like comparing elements may differ.

3. Game Development:

Game Design Frameworks: In game development, the Template Method pattern can be employed in frameworks that define the overall game loop structure. While the main loop remains constant, specific behaviors for updating game objects or handling user input can be customized by individual games.

Character Animation: When designing character animation systems, a common structure for updating and rendering characters can be defined using the Template Method. The specifics of animation sequences can vary for different characters.

4. Educational Software:

Learning Management Systems: Educational software often involves structured processes for presenting content, quizzes, and assessments. A Template Method can define the overall flow, while individual courses or modules provide specific implementations for content delivery and assessments.

5. Document Generation:

Report Generation: In applications generating reports or documents, a Template Method can define the structure of document creation. Specific steps for adding headers, footers, and content generation can be customized based on the type of document.

6. Business Process Automation:

Workflow Management Systems: Systems that automate business processes can benefit from the Template Method pattern. The overall workflow structure remains consistent, but specific steps for handling tasks, approvals, or notifications can be tailored to different business processes.

Pros and Cons

Pros	Cons
Code Reuse: The Template Method pattern promotes code reuse by providing a common structure for algorithms. The overall algorithm is defined in the template method, and individual steps can be reused in subclasses.	**Reduced Runtime Flexibility**: The pattern determines the overall algorithm structure during compile-time, limiting the runtime flexibility of the algorithm. Dynamic changes to the algorithm structure may be challenging.
Promotes Uniform Design: It ensures a consistent algorithm structure across multiple classes or subclasses. This helps maintain a uniform design and facilitates understanding for developers working with the codebase.	**Increased Complexity**: The use of the Template Method can lead to increased complexity in the codebase, especially when dealing with multiple subclasses and variations. Understanding the flow may become challenging as the number of subclasses grows.
Room for Customization: The pattern allows for flexibility by enabling subclasses to customize specific steps of the algorithm. This customization accommodates variations in behavior without disturbing the overall algorithm.	**Reduced Coupling**: The pattern may lead to tight coupling between the template method and its subclasses. Changes in the template method might necessitate modifications in subclasses, impacting the design's extensibility.
Reduced Code Duplication: The Template Method pattern helps reduce code duplication by centralizing the common algorithm structure. Changes or updates to the algorithm can be made in one place, affecting all subclasses.	**Less Effective for Code Modifications**: Handling variations in specific steps of the algorithm can be challenging. The template method might become less effective if variations are extensive and require conditional logic.

In summary, the Template Method pattern offers advantages in terms of code reuse, consistent structure, and flexibility for customization. However, it comes with trade-offs such as increased complexity, potential tight coupling, and limited runtime flexibility. Choosing to use the Template Method pattern should consider the specific requirements and design goals of the application.